项目 5 高层运动控制

1、 预备知识

1.1、 机器人坐标系

机器人的坐标系定义为：X 朝前，Y 朝左，Z 朝上。机器人站立时的默认离地高度为 0.33m，左前足距离机体中心的距离为[x:0.19257, y:0.135]，行走时的抬腿高度 0.09m。

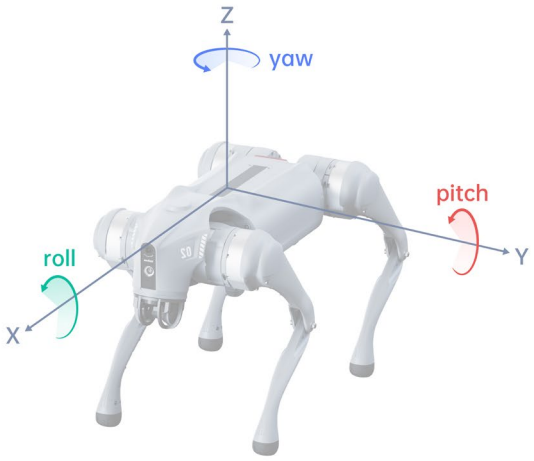


图 1-1 坐标系定义

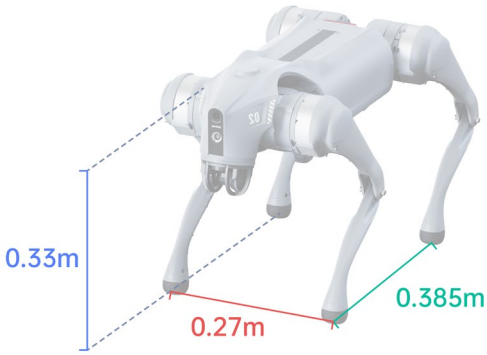


图 1-2 默认的站立状态

1.2、 机器人状态机介绍

机器人的运动包含阻尼、站立锁定、平衡站立、基础运动、特殊动作和保护状态状态，机器人在运行过程中，通过状态机在不同状态之间进行切换，以实现不同的控制任务。

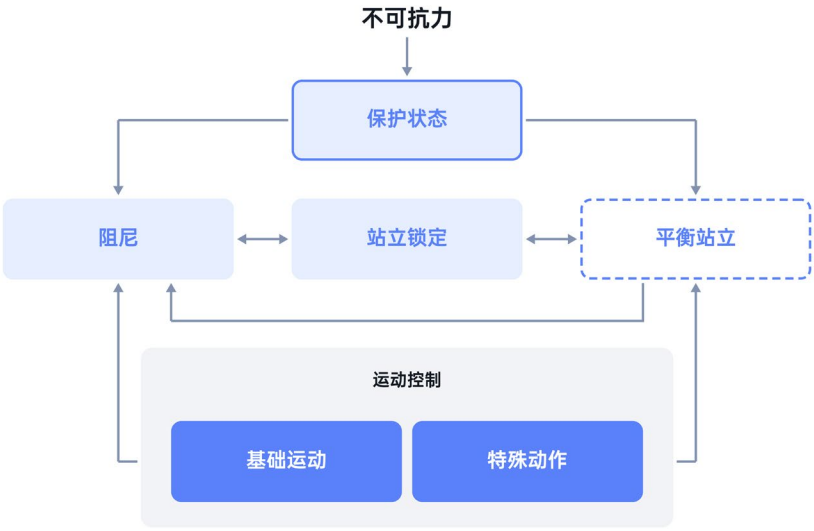


图 1-3 机器人状态切换机制

机器人各个状态的切换机制如上图所示，以下对各个状态具体切换过程进行解释：

阻尼：阻尼状态的切换具有最高优先级，所有状态都可以立刻切换进入阻尼状态，相当于一个软急停机制。

站立锁定：站立锁定是连接阻尼和平衡站立的状态，机器狗需要从阻尼状态进入平衡站立状态后，才能调用相应的运动服务实现机器人的控制。

平衡站立：在平衡站立状态下，可调用 SDK 的各部分接口实现机器人的运动控制。

保护状态：当机器人识别到强烈冲击，并判断出无法正常执行运动动作时，会进入保护状态。SDK 提供了从保护状态恢复至阻尼或站立状态的接口。

基础运动：在运动执行过程中，可调用 SDK 接口，让机器人进入阻尼或平衡站立状态，以终止当前的运动服务。

特殊动作：当进入特殊动作执行状态后，其他运动控制服务会先被挂起，等待当前动作执行完毕并进入平衡站立状态后再生效。

1.3、 运动控制接口

高层运动控制接口可分为运控切换服务接口、高层运动服务接口、AI 运动服务接口。

1.3.1、 运控切换服务接口

运控切换服务接口可以方便地通过 RPC 方式实现对 Go2 运控模式的切换，还可通过接口设置或获取 Go2 是否处于静默状态（Go2 处于静默状态时，Go2 默认运控服务均不启动，但设置成功后在 Go2 重启后生效）。

1.3.2、 高层运动服务接口

机器人的高层运动控制接口服务可分为基础运动控制和特殊动作控制。基础运动控制服务中，可调用相应的接口实现机器人的姿态和机身高度控制、速度控制、轨迹跟踪控制和避障等不同的运动，同时还可以根据不同的地形场景和任务需求调用接口切换机器人的行走步态。特殊动作控制服务中，可调用相应的接口，实现机器人内置的特殊动作，如、坐下、前跳、前空翻等。如下图给出了机器人支持的的高层运动控制接口。

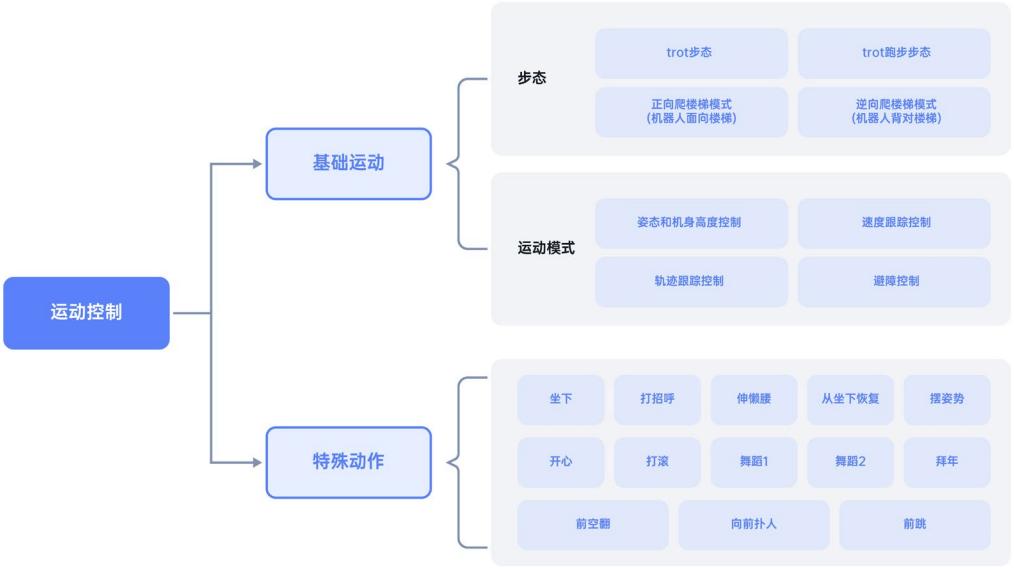


图 1-4 运动空间模式

高层运动服务分为两个部分：高层控制接口、高层状态接口。

高层控制接口：通过调用 SDK 的 sport_client，来给 Go2 发送速度控制、轨迹跟踪等运动指令。

高层状态接口：通过订阅 SDK 中的 HigState 消息，来获取 Go2 的位置、速度、姿态等运动状态。

1.3.3、 AI 运动服务接口

Ai 运动服务，对应的是 Unitree Go App 上的 Ai 模式。为了通过本章节提供的接口控制 Go2 机器人，您首先需要将 Go2 的运控模式切换至 Ai 模式。以下是两种切换模式的方法：

使用 Unitree Go App 切换到 AI 模式。

使用运控切换服务接口，将运控切换到 AI 模式。

2、 高层运动服务应用

高层运动服务分为两个部分：高层控制接口、高层状态接口。

高层控制接口：通过调用 SDK 的 sport_client，来给 Go2 发送速度控制、轨迹跟踪等运动指令。

高层状态接口：通过订阅 SDK 中的 HigState 消息，来获取 Go2 的位置、速度、姿态等运动状态。

2.1、 高层运动接口介绍

高层控制接口可以实现 Go2 的姿态、速度和轨迹跟踪、空翻、跳跃、倒立等控制。下面列出运动控制接口的相关函数。

函数名	Damp
函数原型	int32_t Damp()
功能概述	进入阻尼状态。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	所有电机关节停止运动并进入阻尼状态。该模式具有最高的优先级，用于突发情况下的急停。

函数名	BalanceStand
函数原型	int32_t BalanceStand()
功能概述	解除锁定。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	解除关节电机锁定，从正常站立模式切换到平衡站立模式。这一模式下机身姿态和高度会始终保持平衡，不受地形的影响。可以通过调用 Euler()和 BodyHeight()接口实现机身的字体和高度控制(详见表格的对应部分)。

函数名	RecoveryStand
函数原型	int32_t RecoveryStand()
功能概述	恢复站立。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	从翻倒或趴下状态恢复至平衡站立状态。不论是否翻倒，都会恢复至站立。

函数名	StopMove
函数原型	int32_t StopMove()
功能概述	停下当前动作，将绝大多数指令恢复成默认值。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	停下当前的运动，并将 Go2 内部的运动参数恢复到默认值。

函数名	RecoveryStand
函数原型	int32_t RecoveryStand()
功能概述	恢复站立。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	从翻倒或趴下状态恢复至平衡站立状态。不论是否翻倒，都会恢复至站立。

函数名	StandUp
函数原型	int32_t StandUp()
功能概述	关节锁定，站高。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	机器狗正常站高，电机关节保持锁定。相比于平衡站立模式，该模式下机器狗的姿态不会始终保持平衡。默认的站立高度为 0.33m

函数名	StandDown
函数原型	int32_t StandDown()
功能概述	关节锁定，站低。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	机器狗趴下，电机关节保持锁定。

函数名	Euler
函数原型	int32_t Euler(float roll, float pitch, float yaw)
功能概述	站立和行走时的姿态。
参数	**roll:** 取值范围 [-0.75~0.75] (rad)； **pitch:** 取值范围 [-0.75~0.75] (rad)； **yaw:** 取值范围 [-0.6~0.6] (rad)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	设置 Go2 平衡站立或移动时的机体姿态角。欧拉角采用绕机体相对轴和 z-y-x 旋转顺序的表示方式。

函数名	Move
函数原型	int32_t Move(float vx, float vy, float vyaw)
功能概述	移动。
参数	**vx:** 取值范围[-2.5~3.8] (m/s)； **vy:** 取值范围[-1.0~1.0] (m/s)； **vyaw:** 取值范围[-4~4] (rad/s)。
返回值	调用成功返回 0，否则返回相关错误码。
备注	控制移动速度。设定的速度为机体坐标系表示下的速度。

函数名	Sit
函数原型	int32_t Sit();
功能概述	坐下。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	特殊动作，机器狗坐下。需要注意特殊动作需要在上一个动作执行完毕后再执行，否则会导致动作异常。

函数名	RiseSit
函数原型	int32_t RiseSit()
功能概述	站起（相对于坐下）。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	从坐下状态恢复到平衡站立

函数名	SwitchGait
函数原型	int32_t SwitchGait(int d)
功能概述	切换步态。
参数	**d:** 步态枚举值，取值 0~4，0 为 idle， 1 为 trot，2 为 trot running，3 正向爬楼模式，4：逆向爬楼模式。
返回值	调用成功返回 0，否则返回相关错误码。
备注	正向爬楼模式为机器人头朝楼梯，逆向爬楼模式为机器人背朝楼梯

函数名	BodyHeight
函数原型	int32_t BodyHeight(float height)
功能概述	设置站立和行走时的机身离地高度相对值。
参数	**height：** 与默认机身高度相对值，取值[-0.18~0.03] (m)
返回值	调用成功返回 0，否则返回相关错误码。
备注	调节平衡站立或行走时的机身相对与默认状态的高度，Go2 默认机身高度 0.33。例如： BodyHeight(-0.1)表示调节机身高度至 0.33-0.1=0.23 (m)。

函数名	FootRaiseHeight
函数原型	int32_t FootRaiseHeight(float height)
功能概述	设置移动时的抬足相对高度。
参数	**height：** 与默认抬足高度相对值，取值[-0.06~0.03] (m)
返回值	调用成功返回 0，否则返回相关错误码。
备注	设置 Go2 运动时相对于默认状态的抬腿高度，Go2 默认抬足高度 0.09。例如：FootRaiseHeight(0.01)表示设置抬腿高度为 0.09+0.01=0.1 (m)

函数名	SpeedLevel
函数原型	int32_t SpeedLevel(int level)
功能概述	设置速度档位。
参数	**level**：速度档位枚举值，取值 -1 为慢速，0 为正常，1 为快速。
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Hello
函数原型	int32_t Hello()
功能概述	打招呼。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Stretch
函数原型	int32_t Stretch()
功能概述	伸懒腰。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	TrajectoryFollow
函数原型	int32_t TrajectoryFollow(std::vector path);
功能概述	轨迹跟踪。
参数	path 是机器狗在未来一段时间内的目标运动轨迹，它由 30 个轨迹点 PathPoint 组成。
返回值	调用成功返回 0，否则返回相关错误码。

函数名	SwitchJoystick
函数原型	int32_t SwitchJoystick(bool flag)
功能概述	原生遥控器响应开关。
参数	flag：设置 true 为响应原生遥控器，false 为不响应遥控器。
返回值	调用成功返回 0，否则返回相关错误码。
备注	关闭遥控器响应后，遥控器的指令不会干涉当前程序的运行

函数名	ContinuousGait
函数原型	int32_t ContinuousGait(bool flag)
功能概述	持续移动。
参数	flag：设置 true 为开启，false 为关闭。
返回值	调用成功返回 0，否则返回相关错误码。
备注	启动连续步态后，机器狗会不断保持踏步状态，即使当前速度为 0。

函数名	Wallow
函数原型	int32_t Wallow()
功能概述	打滚。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Pose
函数原型	int32_t Pose(bool flag)
功能概述	摆姿势。
参数	flag：设置 true 为摆姿式，false 为恢复。
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Scrape
函数原型	int32_t Scrape()
功能概述	拜年作揖。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	FrontFlip
函数原型	int32_t FrontFlip()
功能概述	前空翻。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	FrontJump
函数原型	int32_t FrontJump()
功能概述	前跳。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	FrontPounce
函数原型	int32_t FrontPounce()
功能概述	向前扑人。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	Dance1
函数原型	int32_t Dance1()
功能概述	舞蹈段落 1。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

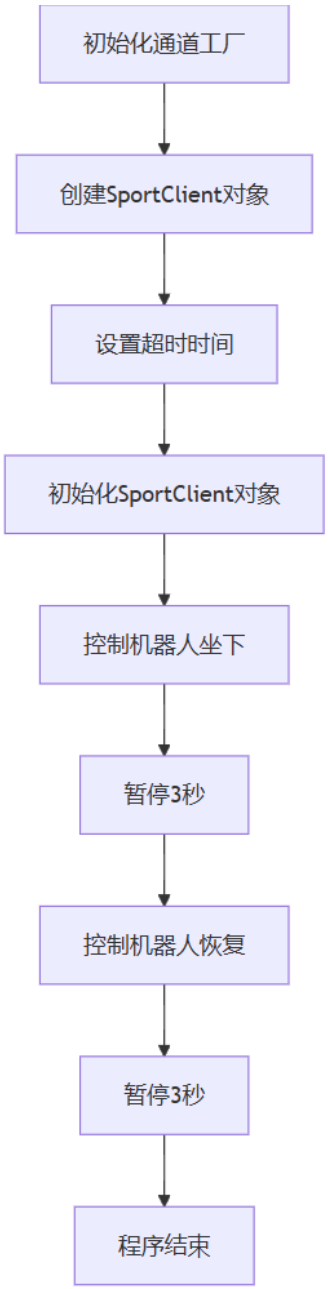
函数名	Dance2
函数原型	int32_t Dance2()
功能概述	舞蹈段落 2。
参数	无
返回值	调用成功返回 0，否则返回相关错误码。
备注	

函数名	GetState
函数原型	int32_t GetState(const std::vectorstd::string& _vector, std::map<std::string, std::string>& _map);
功能概述	获取机器人当前状态，包括： state: 运动状态 bodyHeight：机体高度, footRaiseHeight：抬腿高度, speedLevel：速度档位, gait 步态, joystick： 遥控器状态, dance：是否处于动作中, continuousGait：是否处于连续步态 economicGait：是否处于省电步态
参数	**_vector**：机器人状态名称的vector引用；**_map**： 状态名和对应的值的map引用。
返回值	调用成功返回 0，否则返回相关错误码。

错误号	错误描述
4101	轨迹点数错误，由客户端返回
4201	动作超时错误，在期望的时间内没有完成指定动作
4205	状态机未初始化结束
4206	执行挥手或拜年类动作前，机器人姿态不佳，不予执行
3104	DDS 超时

2.2、 高层运动接口的调用例程

2.2.1、 程序流程图及实现思路



①使用命令行参数初始化通道工厂，创建 SportClient 对象，并设置超时时间。

②使用 SportClient 对象控制机器人执行坐下动作，延时 3 秒；

控制机器人从坐姿恢复。

2.2.2、 完整代码

```
#include <unitree/robot/go2/sport/sport_client.hpp>
#include <unistd.h>
int main(int argc, char **argv)
{
    if (argc < 2)
    {
        std::cout << "Usage: " << argv[0] << " networkInterface" <<
std::endl;
        exit(-1);
    }
    unitree::robot::ChannelFactory::Instance()->Init(0, argv[1]);
    //argv[1]由终端传入，为机器人连接的网卡名称
    //创建 sport client 对象
    unitree::robot::go2::SportClient sport_client;
    sport_client.SetTimeout(10.0f);//超时时间
    sport_client.Init();
```

```
    sport_client.Sit(); //特殊动作，机器狗坐下
    sleep(3);//延迟 3s
    sport_client.RiseSit(); //恢复
    sleep(3);
    return 0;
}
```

2.3、 高层运动状态接口

通过订阅"rt/sportmodestate"话题可以实现对机器人的位置、速度、姿态等运动状态的获取。

2.3.1、 接口说明

获取高层部分状态的方法如下：

```
TimeSpec stamp();
uint32_t errorCode(); //错误代码
IMU imuState(); //IMU 状态
uint8_t mode(); //运动模式
/*
运动模式
0. idle, default stand
```

1. balanceStand

2. pose

3. locomotion

4. reserve

5. lieDown

6. jointLock

7. damping

8. recoveryStand

9. reserve

10. sit

11. frontFlip

12. frontJump

13. frontPounc

*/

float progress(); //是否动作执行状态：0. dance false; 1. dance true

uint8_t gaitType(); //步态类型

/*

步态类型

```
0.idle

1.trot

2.run

3.climb stair

4.forwardDownStair

9.adjust

*/

float footRaiseHeight(); //抬腿高度

std::array<float, 3> position(); //三维位置

float bodyHeight(); //机体高度

std::array<float, 3> velocity(); //三维速度

float yawSpeed(); //偏航速度

std::array<float, 4> rangeObstacle(); //障碍物距离

std::array<int16_t, 4> footForce(); //四个足端力

std::array<float, 12> footPositionBody(); //足端相对于机体的位置

std::array<float, 12> footSpeedBody(); //足端相对与机体的速度

std::array<PathPoint, 10>    pathPoints(); //当前跟踪的路径点
```

其中获取 IMU 数据的方法为：

```
std::array<float, 4> quaternion();        // 四元数 (w,x,y,z)
std::array<float, 3> gyroscope();         // 角速度（unit: rad/s)
std::array<float, 3> accelerometer();     // 加速度 m/(s2)
std::array<float, 3> rpy();               // 欧拉角（unit: rad)
```

PathPoint 的结构为：

```
typedef struct
{
    float tFromStart; //路径点所处时刻
    float x; //x 位置
    float y; //y 位置
    float yaw; //偏航角
    float vx; //x 速度
    float vy; //y 速度
    float vyaw; //偏航速度
} PathPoint;
```

2.3.2、 程序流程及实现思路

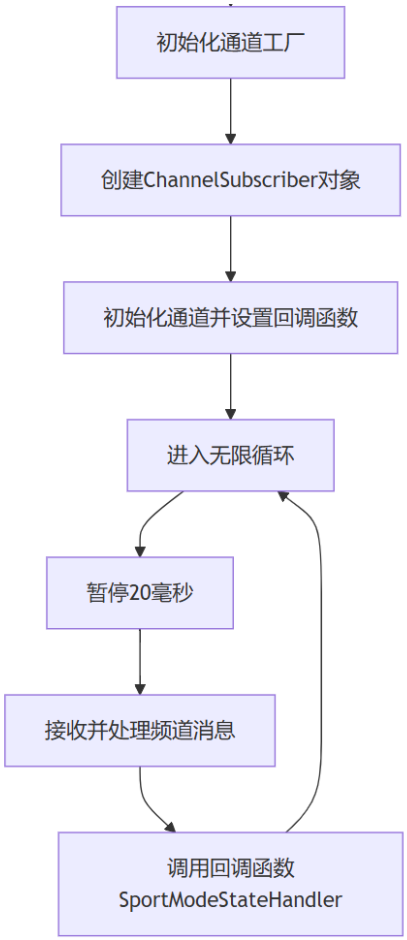
①定义主题 TOPIC_HIGHSTATE，用于订阅机器人的状态信息。定义回调函数 SportModeStateHandler，该函数接收状态信息并打印机器人的位置和 IMU（惯性测量单元）的四元数状态。

②初始化工作：

创建一个 ChannelSubscribr 对象，订阅 TOPIC_HIGHSTATE 频道。

初始化通道并设置回调函数为 SportModeStateHandler。

③进入无限循环，在循环中，等待新的订阅消息。



2.3.3、 完整代码

```
/**
    * @file sportmodestate_test.cpp
    * @brief Subscribe the sportmodestate to get state of robot
    * @date 2023-12-20
    */
#include <unitree/idl/go2/SportModeState_.hpp>
#include <unitree/robot/channel/channel_subscriber.hpp>
//high frequency (500Hz)
#define TOPIC_HIGHSTATE "rt/sportmodestate"
//low frequency (500Hz)
//#define TOPIC_HIGHSTATE "rt/lf/sportmodestate"
using namespace unitree::robot;
//Callback function
void SportModeStateHandler(const void* message)
{
  unitree_go::msg::dds_::SportModeState_ state =
```

```
*(unitree_go::msg::dds_::SportModeState_*)message;
        std::cout<<"position: "<<
                    <<state.position()[0]<<", "
                    <<state.position()[1]<<", "
                    <<state.position()[2]<<std::endl;
        std::cout<<": "<<
                    <<state.imu_state().quaternion()[0]<<", "
                    <<state.imu_state().quaternion()[1]<<", "
                    <<state.imu_state().quaternion()[2]<<", "
                    <<state.imu_state().quaternion()[3]<<std::endl;
    }
    int main()
    {
        //Init
        unitree::robot::ChannelFactory::Instance()->Init();
        //Create a subscriber to get sport mode state
        ChannelSubscriber<unitree_go::msg::dds_::SportModeState_>
sport_mode_state_sub_(TOPIC_HIGHSTATE);
```

```
    //Init channel
    sport_mode_state_sub_.InitChannel(SportModeStateHandler);
    while(1)  {
      usleep(20000);
    }
    return 0;
}
```

3、 项目总结

在本项目中，我们系统地学习并熟练掌握了 Go2 机器人的关键运动控制接口，包括运控切换服务、高层运动服务应用及状态查询。这些接口的深入理解使我们能够灵活地操控 Go2 机器人，实现各种复杂的运动模式。接下来，我们将进入遥控器数据读取项目，进一步探索如何通过遥控器精确控制机器人的行为，为实现更高级的交互和自动化任务奠定基础。